Carleton University
School of Computer Science

Last updated: January 22, 2023

COMP 3000 (WINTER 2023) OPERATING SYSTEMS TUTORIAL 3

Tasks/Questions

1. Compile and run 3000shell.c.

2. Compared to csimpleshell from Tutorial 1, what functionality improvements has 3000shell introduced? List at least two improvements (functional differences). You can mention whether you found them by reading the source code or trying both shells.
   By using both shells, you can see 3000shell shows the current user in the prompt whereas csimpleshell doesn't (with just a "`$`"). Also, if you look at the code, you'll see 3000shell supports additional commands like "`plist`" (although the important command "`cd`" is missing in both). Moreover, you may also notice richer functionalities of 3000shell, such as signal handling, support for `stdout` redirection, execution in the background, etc.

3. Pay attention to lines 229-235. How can you run a program in the background? Note: "in the background" means the shell starting a program without waiting for it to finish. So, the shell returns immediately to be ready to accept the next command.
   As in bash, just use the "`&`" operator at the end.

4. Observe the behavior when running different programs in the background. What happens to the input and output of the program? Try this for different types of programs, such as `ls` and `bc`, or `nano` and `top`, or others of your choice. Write down your observations.
   For non-interactive programs (those taking only inputs from the command line arguments), no difference would be noticed. For interactive ones (e.g., `nano` and `top`) continuously using standard input/output, they may mess up the shared standard input and standard output. If both the parent process and the child process are running (not one being paused by `wait()`), with the shared file descriptors (here mostly standard input), very likely they will interfere. Lines 229-235 decide whether to wait based on `background`. So, `background` will leave both processes running and potentially competing for shared resources (if not properly handled, which is 3000shell's case).

5. You may have trouble interacting with the shell after running programs in the background. How can you recover from such a situation? Note: you can do it in a harsh way for this question. We will see a smarter way next.
   You can hit Ctrl-C to terminate both processes, if the interactive program also handles Ctrl-C. Otherwise, if the terminal is not frozen (but just losing key presses) you can try the interactive program's own way repetitively (e.g., Ctrl-X for `nano`). As the last resort, you can kill them from another terminal using the `kill` command after finding out the PID.

6. 3000shell implements a simple form of output redirection (i.e., the standard output mentioned earlier). What syntax should you use to redirect standard output to a file?

Just `>filename`. Note space after the operator cannot be handled by 3000shell.

7. Make the shell output "Ouch!" when you send it a `SIGUSR1` signal.
   Register for `SIGUSR1` in `main()` with `sigaction()` the same way as other signals. Then, in `signal_handler()`, add an `fprintf() if (the_signal == SIGUSR1)`.

8. If you delete line 324 (`SA_RESTART`), how does the behavior of 3000shell change?
   `SA_RESTART` is to restart/resume the system call interrupted by a signal. In the case of 3000shell, if line 324 was removed, after you send `SIGUSR1`, the command prompt will no longer work because the system call `read()` from the library call `fgets()` cannot be restarted, so it will wait infinitely.

9. Replace the use of `find_env()` with `getenv()`. How do their interfaces differ?
   `getenv()` is neater and takes only one argument: the environment variable name. It searches in the actual environment variables instead of our maintained `*envp[]`.

10. Make `plist` output the parent process id for each process, e.g., "5123 ls (5122)". Pay attention to the `stat` and `status` files in the per-process directories in `/proc`.
    `/proc/<PID>/status` is human-readable and `/proc/<PID>/stat` is intended for machines to read (you need to parse it to get the needed value).

11. If the shell gets stuck as a result of running a program in the background, how can you recover elegantly (resuming it) by sending just a signal (which signal)? Why?
    Send a signal to 3000shell: `SIGCHLD`
    Why: This is like a hack, as this signal is not designed for this purpose. What you need is to make the parent wait on the child as it normally does without the "&" being specified. From the code, there are two places the parent can wait (line 205 and line 234). Look at the function `signal_handler()`, if a signal `SIGCHLD` is received, line 205 will be executed, which puts the parent back to waiting.
    This question shows the importance of understanding the code line by line.